图形学物理仿真 Tutorial for Lab 2

概览

在 Lab 2 中你会实现一个基于FLIP的流体仿真框架。FLIP 是一种混合拉格朗日-欧拉方法（Hybrid Lagrangian-Eulerian method），它结合了粒子（用于平流）和网格（用于求解压力投影）的优势，能够模拟出具有低数值耗散的流体效果。

Demo 要求

1. 支持交互调整时间步长
2. PIC/FLIP 混合实现：支持通过 `flipRatio` 参数在 PIC (Particle-In-Cell) 和 FLIP 之间平滑切换。, 使得 `flipRatio = 0` 时使用**PIC方法**进行模拟， `flipRatio = 1.0` 时使用**FLIP**进行模拟, 而 `flipRatio = 0.95` 则为常用的**FLIP95**。
3. 速度投影 (Incompressibility)：在网格上使用 Gauss-Seidel 迭代法消除速度的散度。

Bonus 进阶挑战 （任选，3次lab的Bonus得分上限为16分）

B1. 可视化与交互增强 ★☆☆

实现更丰富的流体可视化与交互方式，提升可视化效果，帮助理解流体行为。例如：

- 按速度大小 / 密度 / 压强对粒子染色
- 实现流体与**鼠标可控球形障碍物**的交互（拖动、施加速度）

B2. 更精确的压强求解 ★★☆

改进不可压缩求解，提高散度消除效果与数值稳定性：

- 高性能线性系统求解：可以使用 **CG / PCG** 求解泊松方程，建议调用 Eigen 或 Taichi 的 CG 求解函数，如自己实现则难度较大。
- 体积守恒增强 (IDP)：参考 “Implicit Density Projection for Volume Conserving Liquids (SCA/TVCG 2019)”，通过位置修正补偿体积损失，使 FLIP 仿真在长时间模拟下更稳健。 （注：IDP 实现细节可咨询唐一早助教）
- 对比基础Demo中Gauss-Seidel 的速度与效果

B3. 欧拉流体对比 ★★☆

在现有框架之外探索基于网格的方法，理解不同坐标系下描述流体的优劣。

- 网格流体实现：实现Semi-Lagrangian 对流，完成Eulerian流体仿真
- 可选：尝试实现 Advection Reflection 方法，https://jzehnder.me/publications/advectionReflection/
- 对比粒子法（PIC/FLIP）与网格法的求解效果

B4. 实现 APIC ★☆☆ or ★★☆

实现 **APIC（Affine Particle-In-Cell）** 方法，并对比 PIC / FLIP / APIC 的差异 (注：APIC可以咨询张立儒助教)

B5. 表面重建与渲染 ★★★

从粒子生成流体表面：

- 构建表面水平集 / SDF
- 使用 Marching Cubes 提取网格
- 渲染生成的 mesh （光追or辐射度）

B6. 逆向流体优化（前沿挑战）★★★

尝试简单的流体控制 / 反问题，如优化初始条件或外力，使流体达到目标状态，可参考：
https://zhuanlan.zhihu.com/p/527112831

建议与提示

1. 我们建议按照以下的循环函数实现simulation：

```
for (int step = 0; step < numSubSteps; step++) {
    integrateParticles(sdt);
    handleParticleCollisions(obstaclePos, 0.0, obstacleVel);
    if (separateParticles)
        pushParticlesApart(numParticleIters);
    handleParticleCollisions(obstaclePos, 0.0, obstacleVel);
    transferVelocities(true, flipRatio);
    updateParticleDensity();
    solveIncompressibility(numPressureIters, sdt, overRelaxation, compensateDrift);
    transferVelocities(false, flipRatio);
}
```

   - 实现函数 `integrateParticles` 和 `transferVelocities` ，可以观察到粒子受到重力作用而下落。
   - 实现函数 `handleParticleCollisions` 来处理边界条件，以及函数 `solveIncompressibility` 中使用课程中介绍的 `Gauss-Seidel` 方法解决压力问题。
   - 实现 `pushParticlesApart` 函数和 `updateParticleDensity` 函数，以使流体求解更稳定。在 `pushParticlesApart` 函数中，最好使用空间哈希表来加速碰撞检测。
   - 因为3D的FLIP速度相对较慢，我们推荐每个维度的网格分辨率大约是24~32。

2. 基于 VCX 框架： 我们提供了 `FluidSimulator.h` (under vcx-sim-master\src\VCX\Labs\2-FluidSimulation)头文件内容，附以着色器 `fluid.vert` 、 `fluid.frag` (under vcx-sim-master\assets\shaders)作为参考，但你也可以按照自己的想法完成代码。

   - 场景渲染：
     - 如果你选择实现粒子的着色，那么需要在 `CaseFluid::CaseFluid` 的初始化中把 `_program` 的着色器更改为 `fluid.vert` 以及 `fluid.frag` ,同时在 `CaseFluid::OnRender` 函数中更改 `Rendering` 相关设置。

```
Common::CaseRenderResult CaseFluid::OnRender(std::pair<std::uint32_t,
std::uint32_t> const desiredSize) {
    ...
    //change the RenderModel
    Rendering::ModelObject m =
Rendering::ModelObject(_sphere,_simulation.m_particlePos,_simulation.m_particle
Color);
    ...
}
```

- 用户交互：
  - 可以参考Lab0的参考示例，我们希望在这次作业中能够在一定范围内灵活调整时间步长以及 `flipRatio` 。
  - 如果你选择实现移动球形障碍物，你需要更改 `FluidSimulator.h` 文件中的 `obstaclePos` 和 `obstacleVel` ，球形障碍物的生成与渲染可以参考之前 Lab 0 中对流体粒子的实现。

3. 基于 Taichi：
   需要从头构建仿真及交互流程。
   - 并行计算：利用 ti.kernel 编写 map_p2g 和 map_g2p 过程。
   - 可以参考 Taichi 官方文档或其 Gallery 中的 mls_mpm 示例 (python/taichi/examples/simulation/mpm88.py)，但请注意需要实现交错网格，以线性插值完成p2g 和 g2p。

作业提交

作业须提交到**教学网**。

请确保你使用 Git 管理你的代码，在完成 Lab 2 时输出你的提交记录：

```
git log --stat > lab2_log.txt
```

VCX 用户请执行 xmake clean -a ；Taichi 用户请删除 **pycache** 等临时文件夹。

同时，也请提交一份作业报告，简要说明核心代码的思路，实现的交互方法，并展示你的 Demo。

将你输出的提交记录、源代码和报告打包成 zip 压缩包，并以 `lab2_<your-student-ID>.zip` 命名，提交到教学网。

参考资料

Ten Minute Physics: Lecture 18 介绍了2D的FLIP仿真并且提供了参考代码，请确保先自己独立实现！！